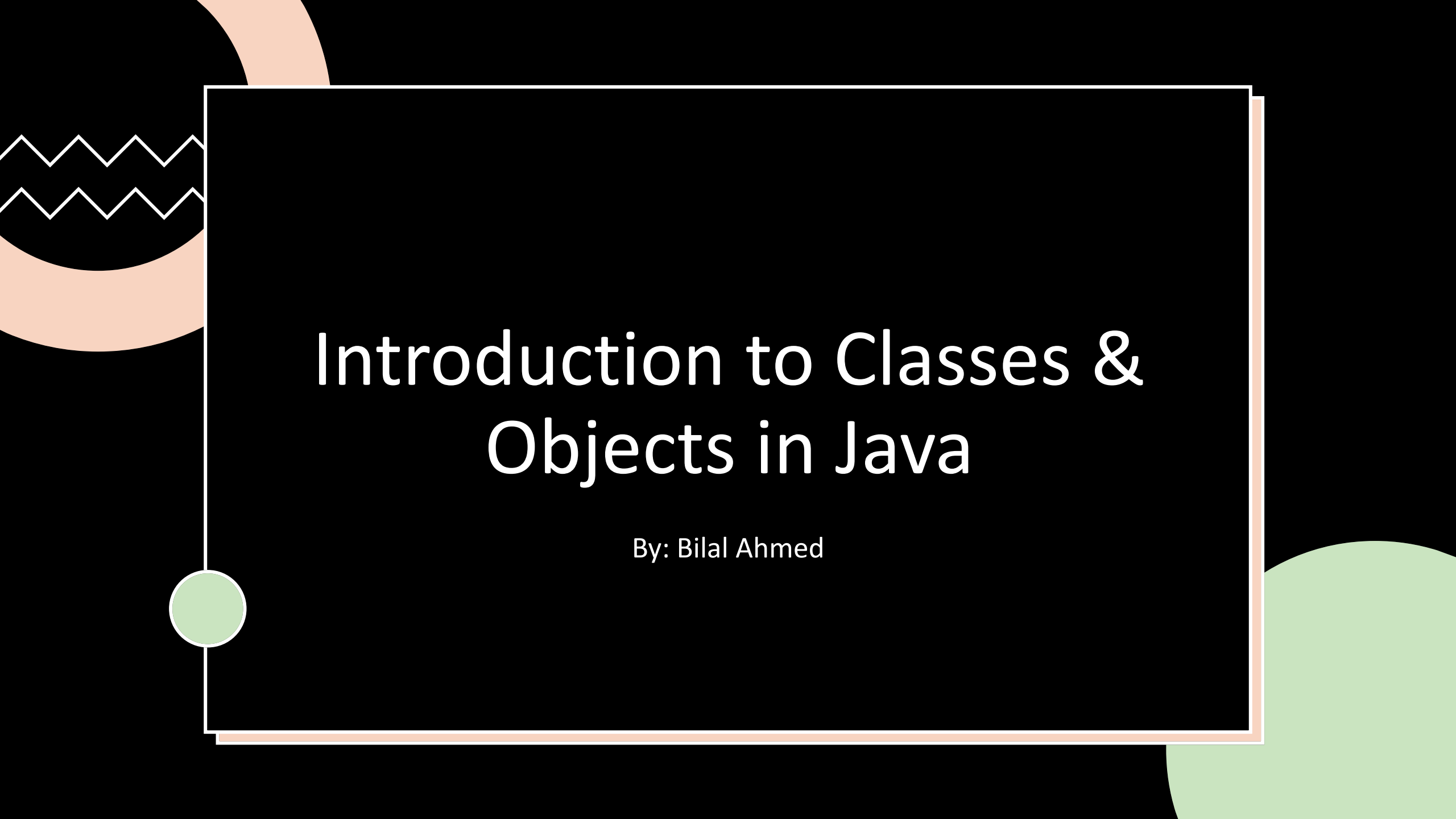


You mastered OOP in C++. But are you sure Java lets you do the same things?



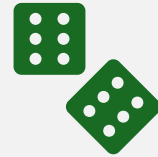
Introduction to Classes & Objects in Java

By: Bilal Ahmed

Class



A **blueprint** for creating objects



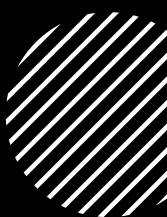
Defines **properties (variables)** and **behaviors (methods)**



Logical structure (no memory until object is created)



Object



A **real-world instance**
of a class

Has **state** (data) and
behavior (methods)

Occupies memory in
the heap



Basic Class Structure



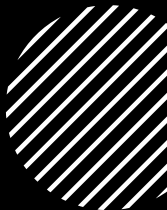
```
class Student {  
    String name; // Instance variable  
    int age;     // Instance variable  
  
    void study() { // Method  
        System.out.println("Studying...");  
    }  
}
```

Method vs Function

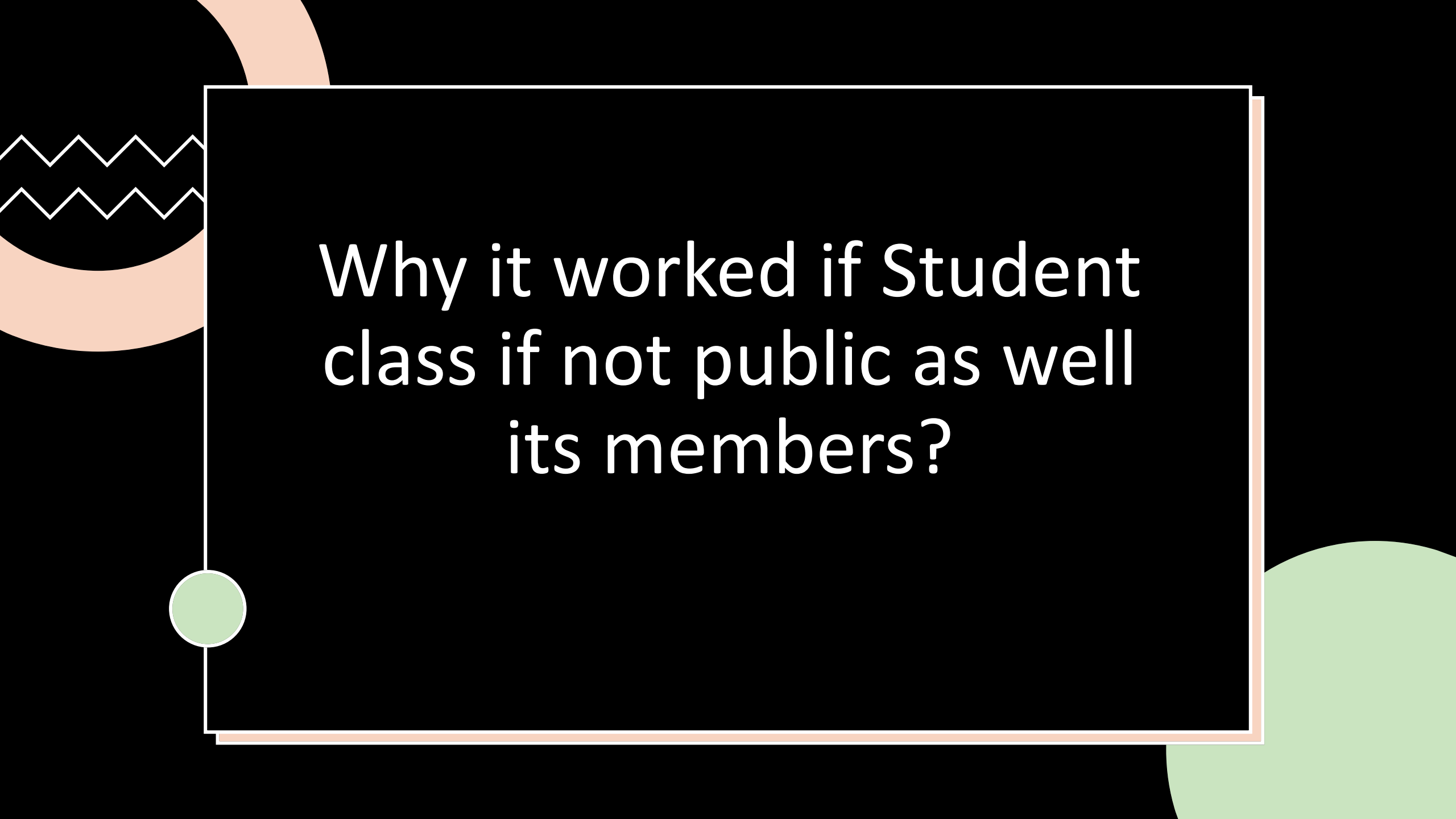
Function	Method
Not associated with objects	Associated with Objects
Invoked through name	Can not invoke by just its name
Independent on class	Dependent on class



Creating an object



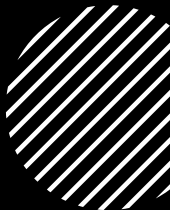
```
public class test {  
    public static void main(String[] args){  
        Student s1 = new Student();  
        s1.name = "Bilal";  
        s1.age = 29;  
        s1.display();  
    }  
}  
class Student {  
    String name; // Instance variable  
    int age;    // Instance variable  
  
    void display() { // Method  
        System.out.println("Name : " + name);  
        System.out.println("Age : " + age);  
    }  
}
```



Why it worked if Student
class is not public as well
its members?



Reasons



test is public (file should be test.java)

Student has **default access (package-private)**

Default access means: Accessible **within the same package (will be covered later)**.

Since both classes are in the same file (and same package), test can access Student.

Why can we access name and age?

-
- These also have **default (package-private) access**.
 - Default access allows:
 - Access inside same class
 - Access inside same package
 - So test can access them because it is in the same package.

When It Would NOT Work

- Student was in another package
- Or members were marked private.
- Because private means: Accessible only inside the same class.

Example:

</> Java

```
class Student {  
    private String name;  
}
```

Now this would give **compile-time error**:

</> Java

```
s1.name = "Bilal";    ❌ Not allowed
```



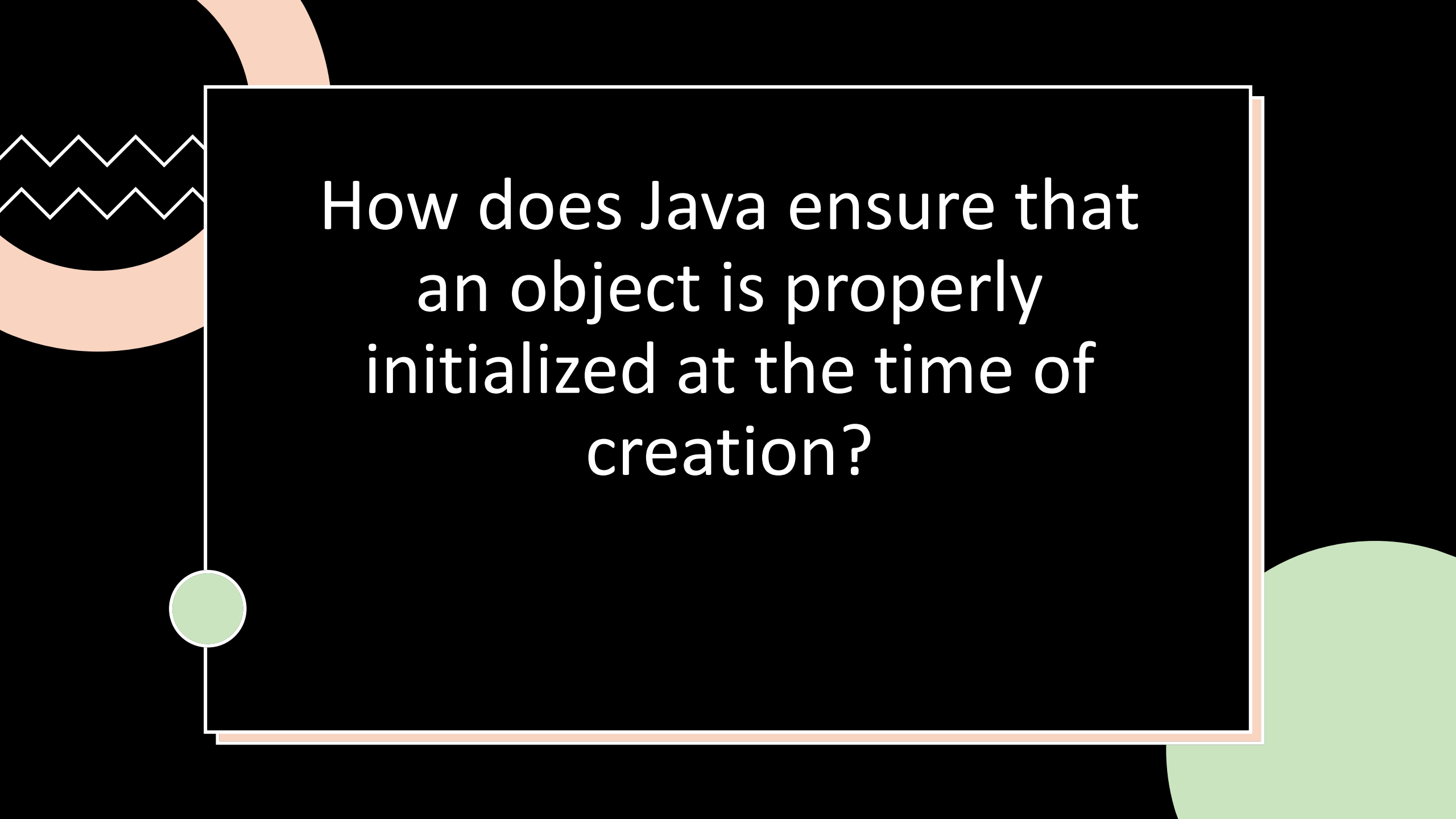
Modifiers in Java



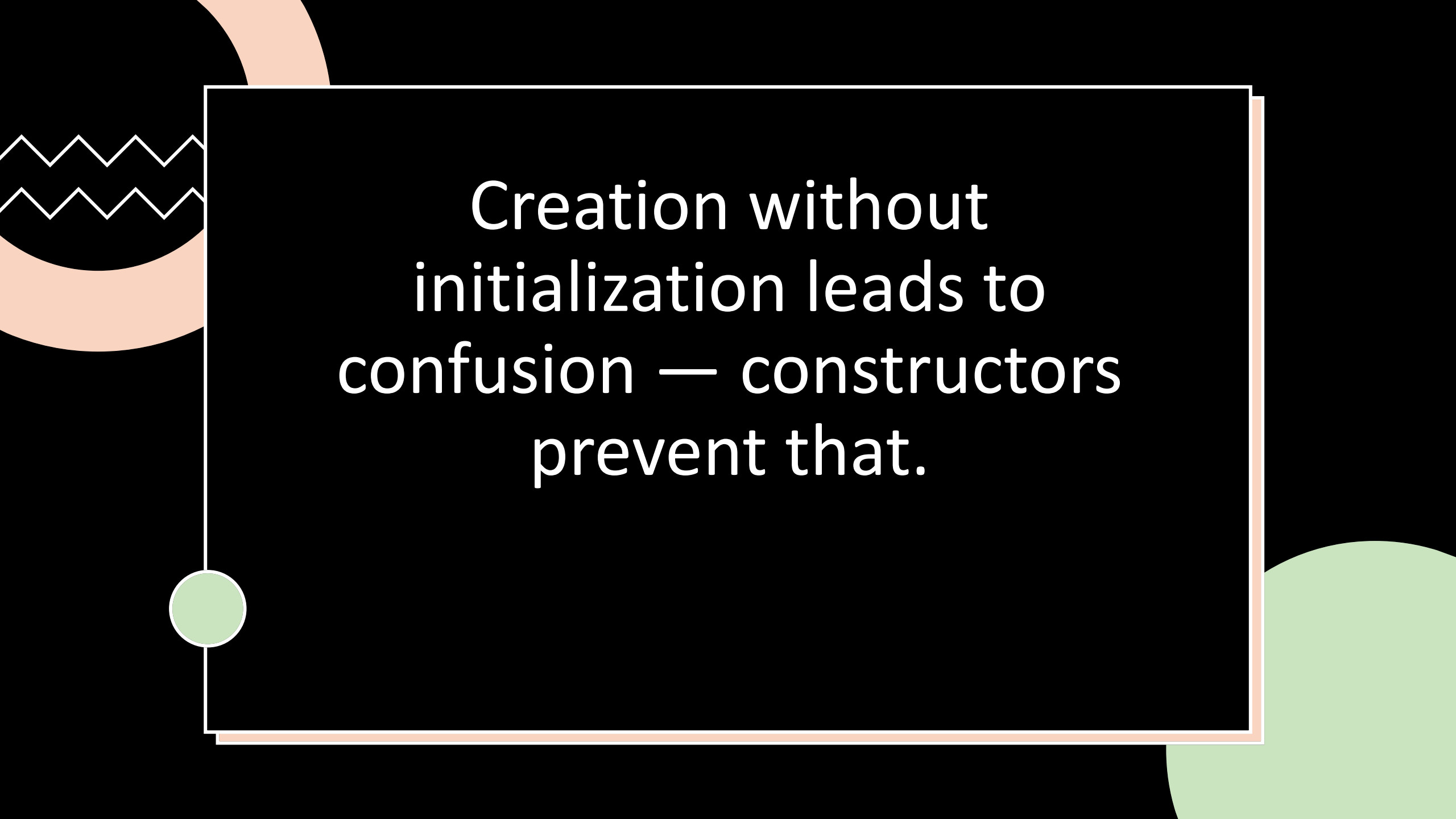
Modifier	Same Class	Same Package	Different Package
private			
(default)			
protected			 (with inheritance)
public			

No Explicit Package

-
- We did not write any package statement.
 - That means our classes are in the **default package**.
 - When two classes are in:
 - The same file → they are in the same package
 - The same folder without package declaration → still same default package
 - That's why default access worked.



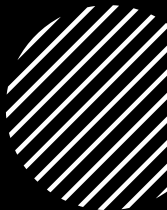
How does Java ensure that
an object is properly
initialized at the time of
creation?



Creation without
initialization leads to
confusion — constructors
prevent that.



What
happens in
the following
code?



```
public class test {  
    public static void main(String[] args){  
        Student s1 = new Student();  
        //s1.name = "Bilal";  
        //s1.age = 29;  
        s1.display();  
    }  
}  
class Student {  
    String name; // Instance variable  
    int age;    // Instance variable  
  
    void display() { // Method  
        System.out.println("Name : " + name);  
        System.out.println("Age : " + age);  
    }  
}
```

Default Constructor Behavior

Object Creation

- An object of Student is created using the new keyword.
- Memory is allocated in the heap.
- The constructor is automatically invoked.

Default Constructor Behavior

- No constructor is explicitly defined in the class.
- Java compiler automatically provides a default (no-argument) constructor.
- This constructor initializes instance variables to default values.

Default Constructor Behavior

Instance Variables

- name and age are instance variables.
- They belong to each object of the class.
- Since no values are assigned manually, they receive default values.

Default Values Assigned

- String → null
- int → 0

I don't like the null and
zero (default values
given by Java), what I
should do?

Custom Default Constructor

```
public class Test {  
    public static void main(String[] args) {  
        Student s1 = new Student(); // Custom default constructor called  
        s1.display();  
    }  
}  
  
class Student {  
    String name;  
    int age;  
  
    // Custom Default Constructor  
    Student() {  
        name = "Unknown";  
        age = 18;  
    }  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}
```

What
happened
in above
code

The programmer defines a constructor with no parameters.

When the object is created, this constructor runs automatically.

Variables are initialized with custom values instead of Java's default values.



What if we want to
initialize the values at
the time of object
creation?



Same
Example
using
parametrize
d

● constructor



```
public class test {  
    public static void main(String[] args){  
        Student s1 = new Student("Bilal",29);  
        s1.display();  
    }  
}  
class Student {  
    String name;  
    int age;  
  
    Student(String n, int a) {  
        name = n;  
        age = a;  
    }  
  
    void display() {  
        System.out.println("Name : " + name);  
        System.out.println("Age : " + age);  
    }  
}
```

What Happens above?

Constructor accepts parameters (String n, int a).

Values are passed during object creation.

Instance variables are initialized with provided values.

Each object can store different data.

Important Rule



If a parameterized constructor is defined and no default constructor is written:



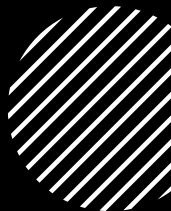
Java **will not** provide a default constructor automatically.



`new Student();` will cause a compile-time error.



Example without custom default constructor



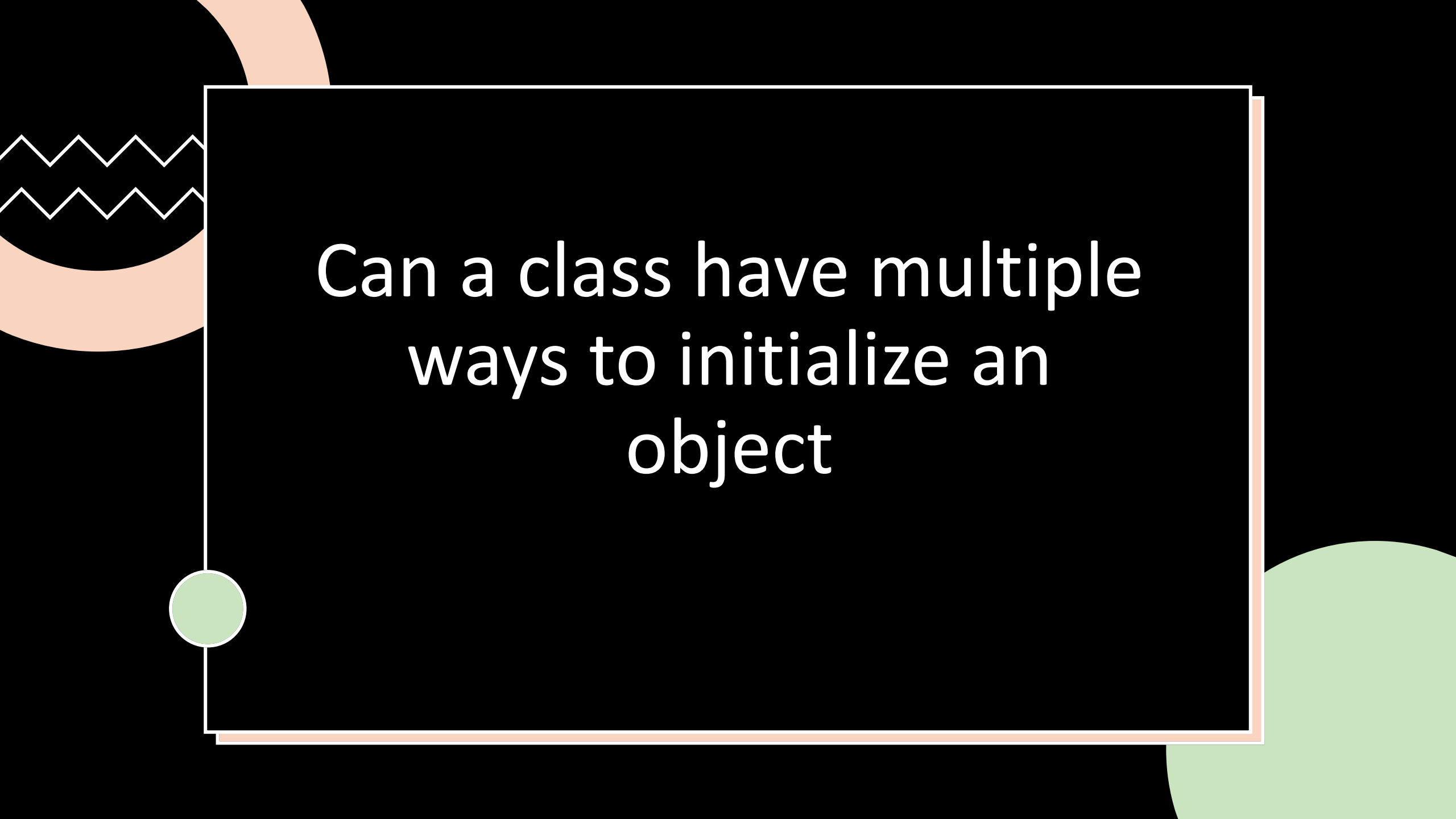
```
public class Test {  
    public static void main(String[] args) {  
        Student s1 = new Student(); // Custom default constructor called  
        s1.display();  
    }  
}  
  
class Student {  
    String name;  
    int age;  
  
    Student(String n, int a) {  
        name = n;  
        age = a;  
    }  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}
```



With both default and parametrize d constructor



```
public class Test {  
    public static void main(String[] args) {  
        Student s1 = new Student(); // Custom default constructor called  
        s1.display();  
    }  
}  
  
class Student {  
    String name;  
    int age;  
  
    // Paramertized Constructor  
    Student(String n, int a) {  
        name = n;  
        age = a;  
    }  
    // Custom Default Constructor  
    Student () {  
        name = "unknown";  
        age = 18;  
    }  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}
```



Can a class have multiple
ways to initialize an
object

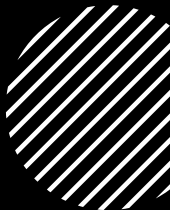
Constructor Overloading

—

Defining **multiple constructors** in the same class with different parameters



Purpose



To provide multiple ways to initialize objects



To increase flexibility in object creation



To support different initialization scenarios



Example



```
public class Test {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        Student s2 = new Student("Bilal");  
        Student s3 = new Student("Bilal", 29);  
  
        s1.display();  
        s2.display();  
        s3.display();  
    }  
}  
  
class Student {  
    String name;  
    int age;  
  
    // Default Constructor  
    Student() {  
        name = "Unknown";  
        age = 0;  
    }  
  
    // Constructor with one parameter  
    Student(String n) {  
        name = n;  
        age = 0;  
    }  
  
    // Constructor with two parameters  
    Student(String n, int a) {  
        name = n;  
        age = a;  
    }  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
        System.out.println();  
    }  
}
```

What Makes It Overloading?

- Same constructor name (Student)
- Different parameter lists
- Different number or type of parameters

Output

Name: Unknown

Age: 0

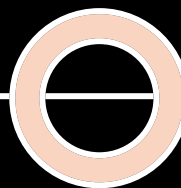
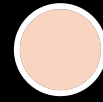
Name: Bilal

Age: 0

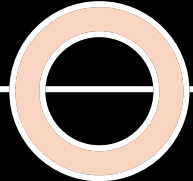
Name: Bilal

Age: 29

=== Code Execution Successful ===



What happens when
constructor
parameters and
instance variables have
the same name?



How does Java
differentiate
between local
variables and
instance variables?



Problem

```
class Student {  
    String name;  
    int age;  
  
    Student(String name, int age) {  
    }  
}
```

-
- When parameter names and instance variable names are the same:
 - Java gets confused between:
 - name → parameter
 - name → instance variable

What will happen?

```
public class Test {  
    public static void main(String[] args) {  
        Student s1 = new Student("Bilal", 29);  
        s1.display();  
    }  
}  
  
class Student {  
    String name;  
    int age;  
  
    Student(String name, int age) {  
        name = name;  
        age = age;  
    }  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}
```

This keyword



this removes this ambiguity.



The *this* keyword refers to the **current object** of the class.



It is used to distinguish between:
Instance variables and Local
variables (parameters)

Example

```
class Student {  
    String name;  
    int age;  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}
```

What
Happens
above?

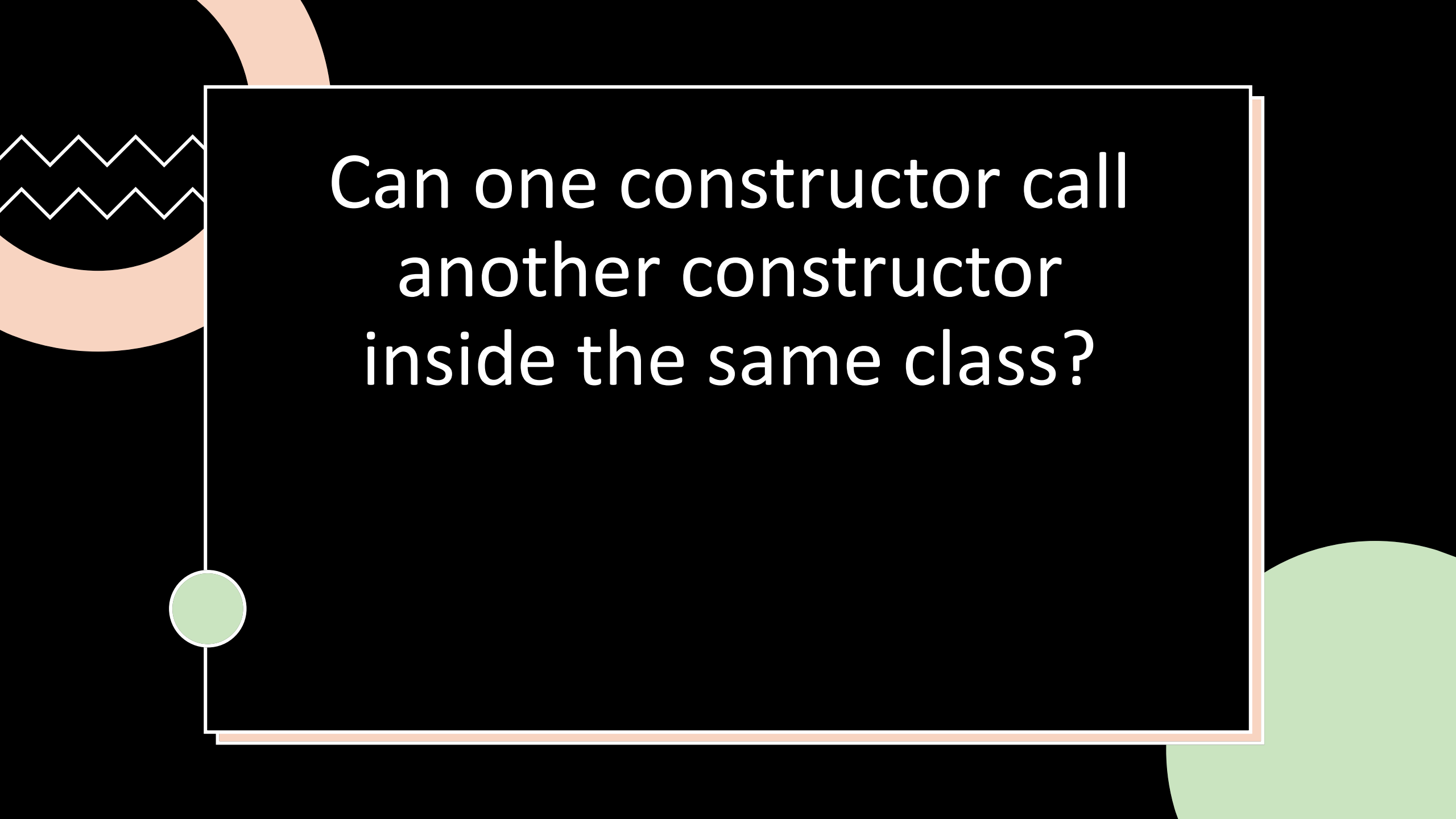
`this.name` → refers to
the instance variable

`name` → refers to the
constructor parameter

Values are assigned
correctly to the object

Working Example

```
public class Test {  
    public static void main(String[] args) {  
        Student s1 = new Student("Bilal", 29);  
        s1.display();  
    }  
}  
  
class Student {  
    String name;  
    int age;  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}
```



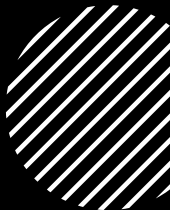
Can one constructor call
another constructor
inside the same class?

Constructor Chaining using this()

Constructor chaining means calling **one constructor from another constructor** within the same class using the this() keyword.



Constructor Chaining using this()



Features

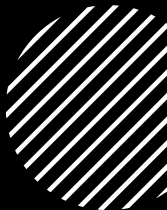
- Code reuse
- Reducing duplication
- Centralizing initialization logic

Rules

- this() must be the **first statement** inside the constructor.
- It calls another constructor of the same class.
- It is used for constructor-to-constructor calling.



Example



```
public class Test {  
    public static void main(String[] args) {  
        Student s1 = new Student("Bilal",29);  
        Student s2 = new Student();  
        s1.display();  
        s2.display();  
    }  
}  
  
class Student {  
    String name;  
    int age;  
  
    // Default Constructor  
    Student() {  
        this("Unknown", 0); // Calls parameterized constructor  
    }  
  
    // Parameterized Constructor  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}
```



What Happens Here?



Student() calls Student(String, int) using this().

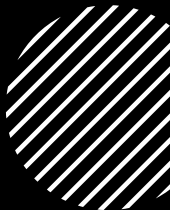
Initialization logic is written only once.

Code duplication is avoided.

Object gets initialized through chained constructors.



Example 2 (without main class)



```
class Student {
    String name;
    int age;

    // Default Constructor
    Student() {
        this("Unknown", 0); // Calls parameterized constructor
    }

    Student(String name) {
        this(name, 0); // Calls parameterized constructor
    }

    Student(int age) {
        this("unknown", 18); // Calls parameterized constructor
    }

    // Parameterized Constructor
    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
```

Summary



Class & Object → Objects are created from a class and stored in heap memory.



Default Constructor → Automatically provided if not defined; initializes variables with default values.



Custom & Parameterized Constructors → Used to initialize objects with meaningful or user-defined values.



Constructor Overloading → Multiple constructors in a class with different parameters for flexible object creation.



this & this() → this refers to the current object; this() enables constructor chaining for code reuse.